PROYECTO INTEGRAL DE DESARROLLO SPA:
ANGULAR E INTEGRACIÓN DE SERVICIOS REST

Manual Práctico Paso a Paso: Construcción de una SPA con HttpClient, RxJS, Interceptores, Guards y Angular Router

SECCIÓN 1: ARQUITECTURA DEL PROYECTO Y OBJETIVOS

Propósito del Proyecto: Este proyecto práctico tiene como objetivo desarrollar una Single Page Application (SPA) para la Gestión de Tareas y Proyectos Empresariales en Angular, demostrando la integración limpia de servicios HTTP asíncronos, la programación reactiva con RxJS, el manejo centralizado de errores, la interceptación de solicitudes y la protección de rutas mediante Guards.

Estructura de la Arquitectura: La arquitectura de la aplicación se divide en las siguientes capas de responsabilidad:
1. Capa de Modelos (Interfaces TypeScript): Define la estructura fuertemente tipada para los objetos de la API.
2. Capa de Servicios HTTP (HttpClient & RxJS): Encapsula las llamadas a endpoints RESTful, manejo de errores con catchError y estado compartido.
3. Capa de Interceptores (HttpInterceptor): Inyecta cabeceras de autorización de forma centralizada a todas las peticiones salientes.
4. Capa de Seguridad y Routing (Guards & Router): Protege las vistas privadas mediante CanActivate y gestiona la navegación cliente sin recargas.
5. Capa de Presentación (Componentes & Pipe Async): Muestra la interfaz y se suscribe de forma reactiva a las fuentes de datos.

SECCIÓN 2: CREACIÓN Y CONFIGURACIÓN DEL PROYECTO

1. Herramientas Base: Verifica la instalación de Node.js e instala Angular CLI si aún no lo has hecho:

```
// --- Terminal ---
npm install -g @angular/cli
```

2. Creación del Proyecto Angular: Crea el nuevo proyecto Angular con enrutamiento activo e ingresa al directorio de trabajo:

```
// --- Terminal ---
ng new spa-servicios-demo --routing --style=css
cd spa-servicios-demo
```

SECCIÓN 3: CAPA DE MODELOS (TypeScript)

3.1 Interfaz Tarea (tarea.model.ts): En la carpeta 'src/app/models/' definimos las interfaces 'tarea.model.ts' para garantizar la seguridad de tipos:

```
// --- src/app/models/tarea.model.ts ---
// src/app/models/tarea.model.ts

export interface Tarea {
  id?: number;
  titulo: string;
  descripcion: string;
  completada: boolean;
  prioridad: 'Alta' | 'Media' | 'Baja';
}
```

SECCIÓN 4: CAPA DE SERVICIO HTTP Y RXJS

4.1 Servicio de Tareas (tarea.service.ts): Genera el servicio 'ng generate service services/tarea'. El servicio utiliza HttpClient, los operadores de RxJS (catchError, throwError, of) y simula consumo REST con resiliencia a errores:

```
// --- src/app/services/tarea.service.ts ---
// src/app/services/tarea.service.ts

import { Injectable } from '@angular/core';
import { HttpClient, HttpErrorResponse } from '@angular/common/http';
import { Observable, of, throwError } from 'rxjs';
import { catchError, retry } from 'rxjs/operators';
import { Tarea } from '../models/tarea.model';

@Injectable({
  providedIn: 'root'
})
public class TareaService {

  private apiUrl = 'https://jsonplaceholder.typicode.com/todos'; // API de prueba

  // Estado local simulado para respaldo
```

```
  private tareasMemoria: Tarea[] = [
    { id: 1, titulo: 'Diseñar Arquitectura SPA', descripcion: 'Definir rutas y
servicios', completada: true, prioridad: 'Alta' },
    { id: 2, titulo: 'Integrar HttpClient', descripcion: 'Conectar con API REST y
RxJS', completada: false, prioridad: 'Alta' }
  ];

  constructor(private http: HttpClient) { }

  getTareas(): Observable<Tarea[]> {
    return of(this.tareasMemoria).pipe(
      retry(1),
      catchError(this.manejarError)
    );
  }

  agregarTarea(tarea: Tarea): Observable<Tarea> {
    tarea.id = this.tareasMemoria.length + 1;
    this.tareasMemoria.push(tarea);
    return of(tarea);
  }

  eliminarTarea(id: number): Observable<boolean> {
    this.tareasMemoria = this.tareasMemoria.filter(t => t.id !== id);
    return of(true);
  }

  private manejarError(error: HttpErrorResponse) {
    console.error('Error en la comunicación HTTP:', error.message);
    return throwError(() => new Error('Error en el servicio REST. Intente más
tarde.'));
  }
}
```

SECCIÓN 5: CAPA DE INTERCEPTORES Y SEGURIDAD (Guards)

5.1 Interceptor HTTP (auth.interceptor.ts): Crea un interceptor con 'ng generate interceptor interceptors/auth' para inyectar cabeceras de autorización de forma centralizada:

```
// --- src/app/interceptors/auth.interceptor.ts ---
// src/app/interceptors/auth.interceptor.ts

import { Injectable } from '@angular/core';
import { HttpRequest, HttpHandler, HttpEvent, HttpInterceptor } from
```

```
'@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
public class AuthInterceptor implements HttpInterceptor {

  intercept(request: HttpRequest<unknown>, next: HttpHandler):
Observable<HttpEvent<unknown>> {
    const token = 'TOKEN_JWT_DE_PRUEBA_12345';

    // Clona la petición e inyecta la cabecera de autorización
    const authReq = request.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });

    return next.handle(authReq);
  }
}
```

5.2 Guard de Seguridad (auth.guard.ts): Crea el Guard 'ng generate guard guards/auth' para proteger rutas privadas de la SPA:

```
// --- src/app/guards/auth.guard.ts ---
// src/app/guards/auth.guard.ts

import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';

@Injectable({
  providedIn: 'root'
})
public class AuthGuard implements CanActivate {

  constructor(private router: Router) {}

  canActivate(): boolean {
    const usuarioAutenticado = true; // Simulación de sesión activa

    if (usuarioAutenticado) {
      return true;
    } else {
      this.router.navigate(['/login']);
      return false;
    }
  }
```

```
}
```

SECCIÓN 6: COMPONENTE LISTADO Y PIPE ASYNC

6.1 Clase del Componente (tarea-lista.component.ts): Genera el componente 'ng generate component components/tarea-lista'. Inyecta el servicio y expone un Observable para ser consumido por el Pipe Async:

```
// --- src/app/components/tarea-lista/tarea-lista.component.ts ---
// src/app/components/tarea-lista/tarea-lista.component.ts

import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { Tarea } from '../../models/tarea.model';
import { TareaService } from '../../services/tarea.service';

@Component({
  selector: 'app-tarea-lista',
  templateUrl: './tarea-lista.component.html',
  styleUrls: ['./tarea-lista.component.css']
})
public class TareaListaComponent implements OnInit {

  tareas$!: Observable<Tarea[]>;

  constructor(private tareaService: TareaService) { }

  ngOnInit(): void {
    this.cargarTareas();
  }

  cargarTareas(): void {
    this.tareas$ = this.tareaService.getTareas();
  }

  eliminar(id: number | undefined): void {
    if (id) {
      this.tareaService.eliminarTarea(id).subscribe(() => {
        this.cargarTareas();
      });
    }
  }
}
```

6.2 Plantilla HTML con Pipe Async (tarea-lista.component.html): Plantilla HTML que utiliza el Pipe Async (| async) para realizar la suscripción y des-suscripción automática:

```
// --- src/app/components/tarea-lista/tarea-lista.component.html ---
<!-- src/app/components/tarea-lista/tarea-lista.component.html -->

<div class="container">
  <h2>📋 Lista de Tareas (Consumo Reactivo con Pipe Async)</h2>

  <div *ngIf="tareas$ | async as tareas; else cargando">
    <table class="tabla-spa">
      <thead>
        <tr>
          <th>ID</th>
          <th>Título</th>
          <th>Descripción</th>
          <th>Prioridad</th>
          <th>Estado</th>
          <th>Acción</th>
        </tr>
      </thead>
      <tbody>
        <tr *ngFor="let t of tareas">
          <td>{{ t.id }}</td>
          <td>{{ t.titulo }}</td>
          <td>{{ t.descripcion }}</td>
          <td>{{ t.prioridad }}</td>
          <td>{{ t.completada ? '✅ Completada' : '⏳ Pendiente' }}</td>
          <td>
            <button (click)="eliminar(t.id)" class="btn-borrar">Eliminar</button>
          </td>
        </tr>
      </tbody>
    </table>
  </div>

  <ng-template #cargando>
    <p class="cargando-text">Cargando datos desde el Servicio REST...</p>
  </ng-template>
</div>
```

SECCIÓN 7: CONFIGURACIÓN DE RUTAS Y MAIN LAYOUT

7.1 Rutas de la SPA (app-routing.module.ts): Configura el enrutador de la SPA en 'app-routing.module.ts' incorporando el Guard de seguridad:

```
// --- src/app/app-routing.module.ts ---
// src/app/app-routing.module.ts

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { TareaListaComponent } from './components/tarea-lista/tarea-lista.component';
import { AuthGuard } from './guards/auth.guard';

const routes: Routes = [
  { path: '', redirectTo: '/tareas', pathMatch: 'full' },
  { path: 'tareas', component: TareaListaComponent, canActivate: [AuthGuard] },
  { path: '**', redirectTo: '/tareas' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

7.2 Layout Principal (app.component.html): Configura la plantilla raíz 'app.component.html' con la etiqueta '<router-outlet>':

```
// --- src/app/app.component.html ---
<!-- src/app/app.component.html -->

<header class="header-spa">
  <h1>⚡ Dashboard SPA - Angular & Servicios REST</h1>
</header>

<main class="main-body">
  <!-- Contenedor dinámico de rutas cliente -->
  <router-outlet></router-outlet>
</main>
```

SECCIÓN 8: EJECUCIÓN Y PRUEBAS DEL PROYECTO

Pasos de Verificación: 1. Iniciar Servidor: Ejecuta 'ng serve --open' en tu consola para compilar la aplicación e iniciar el servidor local en http://localhost:4200.
2. Probar Navegación SPA: Navega por la aplicación y comprueba cómo la vista se actualiza sin recargas de página ni parpadeos.
3. Comprobar Pipe Async y RxJS: Observa la carga asíncrona de datos desde el servicio a través del flujo de datos del Pipe Async.
4. Verificar Interceptor y Guard: Comprueba en la pestaña 'Red' (Network) de las herramientas de

desarrollo del navegador que la cabecera 'Authorization' se incluye en las solicitudes HTTP.

Conclusión Técnica: Este proyecto demuestra cómo integrar el consumo de servicios REST en una Single Page Application con Angular, aplicando programación reactiva con RxJS, interceptores de red, guards de seguridad y el pipe async para lograr aplicaciones web rápidas, limpias y escalables.